Tema 5. SQL avanzado

Índice

1.- Gestión de usuarios....2
1.1.- Creación de usuarios....2
1.2.- Borrado de usuarios....2
1.3.- Asignación de privilegios a usuarios....2
1.4.- Eliminación de permisos....3
1.5.- Consultar usuarios de la base de datos....3
1.6.- Mostrar privilegios de un usuario....3
2.- Transacciones....3
2.1.- Transacciones MySQL....4
3.- Procedimientos almacenados y funciones....5
3.1.- Procedimientos almacenados....5
3.2.- Delimitador MySQL....6
3.3.- Variables....7
3.4.- Parámetros....9
3.5.- Funciones almacenadas....10
3.6.- Instrucciones condicionales....11
3.8.- Cursores....13
4.-Triggers....16
4.1.- Consideraciones....17
4.2.- Uso de triggers....17

En esta unidad, estudiaremos diferentes aspectos avanzados de SQL, como gestión de usuario, transacciones, procedimientos almacenados y activadores.

1.- Gestión de usuarios

La seguridad y la administración de los usuarios es un componente crítico en la gestión de bases de datos. SQL permite la creación y manejo de usuarios y roles para definir los privilegios de acceso a los diferentes objetos y datos en una base de datos.

1.1.- Creación de usuarios

La creación de usuarios en SQL se realiza con el comando CREATE USER. A continuación, se muestra un ejemplo:

```
CREATE USER 'nombre_usuario'@'localhost' IDENTIFIED BY 'contrasenia_usuario';
```

Aquí, 'nombre_usuario' es el nombre del usuario y 'contraseña_usuario' es su contraseña. Usar 'localhost' restringe la conexión al usuario desde la misma máquina donde reside la base de datos. Para permitir conexiones desde cualquier lugar, se puede reemplazar 'localhost' con '%'.

1.2.- Borrado de usuarios

Para eliminar un usuario usamos DROP USER.

```
DROP USER 'nombre_usuario'@'localhost';
```

1.3.- Asignación de privilegios a usuarios

Una vez creado el usuario, es posible asignarle privilegios específicos utilizando el comando GRANT. Este comando permite determinar qué acciones puede realizar el usuario (selección, inserción, actualización, eliminación, etc.) y sobre qué bases de datos o tablas puede realizarlas. A continuación, se muestra un ejemplo de cómo otorgar todos los privilegios a un usuario sobre una base de datos específica:

```
GRANT ALL PRIVILEGES ON nombre_base_datos.* TO 'nombre_usuario'@'localhost';
```

En este caso, nombre_base_datos es el nombre de la base de datos a la que se le están otorgando los privilegios.

Es crucial asignar solo los privilegios necesarios para minimizar riesgos de seguridad.

Si queremos asignar permisos SELECT e INSERT en una tabla específica

```
GRANT SELECT, INSERT ON empresa.clientes TO 'nombre_usuario'@'localhost';
```

1.4.- Eliminación de permisos

Para quitar permisos, puedes usar el comando REVOKE. Por ejemplo:

```
REVOKE INSERT ON empresa.clientes FROM 'nombre_usuario'@'localhost';
```

1.5.- Consultar usuarios de la base de datos

Esta consulta muestra los permisos específicos para cada usuario en las bases de datos a las que tienen acceso.

```
SELECT
    user AS UserName,
    host AS Host,
    db AS DatabaseName,
    Select_priv,
    Insert_priv,
    Update_priv,
    Delete_priv,
    Create_priv,
    Alter_priv
FROM mysql.user;
```

1.6.- Mostrar privilegios de un usuario

Si deseas ver los permisos en una base de datos específica, puedes ajustar la consulta según tus necesidades:

```
SHOW GRANTS FOR 'nombre_usuario'@'localhost';
```

Esto mostrará los permisos asignados al usuario ‘nombre_usuario’ desde la dirección 'localhost' en la base de datos especificada

2.- Transacciones

Una transacción es una unidad lógica de trabajo que contiene una o más sentencias SQL. Las transacciones son unidades atómicas de trabajo.

Una transacción es la propagación de uno o más cambios a la base de datos. Si todos los cambios tienen éxito, la transacción se confirma. Pero si hay algún problema, todos los cambios se deshacen cuando se revierte la transacción.

- Una transacción comienza con la primera sentencia SQL ejecutable o explícitamente con START TRANSACTION.

- Una transacción finaliza cuando se confirma o se revierte, con una sentencia COMMIT o ROLLBACK.

Las sentencias DDL (Data Definition Lenguage) no se puede revertir e implícitamente finaliza la transacción.

Para entender el concepto de transacción, veamos un ejemplo de una base de datos bancaria. Supongamos que un cliente bancario transfiere dinero de su cuenta de ahorros a su cuenta corriente, el extracto se dividirá en cuatro bloques:

- Iniciar transacción
- Cambiar el saldo en la cuenta de origen.
- Cambiar el saldo en la cuenta de destino.
- Finalizar transacción La sentencia SQL para iniciar la transacción:

La declaración SQL para iniciar la transacción:

```
START TRANSACTION
```

La sentencia SQL para cambiar el saldo en la cuenta de origen:

```
UPDATE Cuentas SET balance = balance - 1000
WHERE numeroCuenta = '932656' ;
```

La sentencia SQL para cambiar el saldo en la cuenta de destino:

```
UPDATE Cuentas SET balance = balance + 1000
WHERE numeroCuenta = '102656' ;
```

La declaración SQL para finalizar una transacción es:

```
COMMIT;
```

2.1.- Transacciones MySQL

Estas instrucciones proporcionan control sobre el uso de las transacciones:

- START TRANSACTION o BEGIN inician una nueva transacción.
- COMMIT confirma la transacción actual y hace que sus cambios sean permanentes.
- ROLLBACK revierte la transacción actual y cancela sus cambios.
- SET autocommit deshabilita o habilita el modo de confirmación automática predeterminado para la sesión actual.

- SET autocommit=0;
- SET autocommit=1;

De manera predeterminada, MySQL se ejecuta con el modo de confirmación automática habilitado. Esto significa que, cuando una oración no está dentro de una transacción, cada instrucción es atómica, como si estuviera rodeada por START TRANSACTION y COMMIT. No puede usar ROLLBACK para deshacer el efecto; sin embargo, si ocurre un error durante la ejecución de la instrucción, la instrucción se revierte.

3.- Procedimientos almacenados y funciones

3.1.- Procedimientos almacenados

Un procedimiento almacenado es un segmento de sentencias SQL declarativas almacenadas dentro del servidor MySQL.

En este ejemplo, hemos creado un procedimiento almacenado con el nombre GetClientes()

```
DELIMITER $$
CREATE PROCEDURE GetClientes()
BEGIN
    SELECT nombreCliente, ciudad, estado, codigoPostal, pais
    FROM Clientes ORDER BY nombreCliente;
END$$
DELIMITER ;
```

Una vez que guarde el procedimiento almacenado, puede invocarlo mediante la instrucción CALL

```
CALL GetClientes()
```

Y la declaración devuelve el mismo resultado que la consulta.

La primera vez que invoca un procedimiento almacenado, MySQL compila el código del procedimiento almacenado, lo coloca en un área de memoria conocida como caché y ejecuta el procedimiento almacenado.

Si invoca el mismo procedimiento almacenado en la misma sesión nuevamente, MySQL simplemente ejecuta el procedimiento almacenado desde la caché sin tener que volver a compilarlo.

Un procedimiento almacenado puede tener parámetros para que pueda pasarle valores y obtener el resultado. Por ejemplo, puede tener un procedimiento almacenado que devuelva clientes por país y ciudad. En este caso, el país y la ciudad son parámetros del procedimiento almacenado.

Un procedimiento almacenado puede contener declaraciones de flujo de control como IF, CASE y LOOP que le permiten implementar el código de manera procedimental.

Un procedimiento almacenado puede llamar a otros procedimientos almacenados o funciones almacenadas, lo que le permite modularizar su código.

3.2.- Delimitador MySQL

Al escribir sentencias SQL, se utiliza el punto y coma (;) para separar dos sentencias como en el siguiente ejemplo:

```
SELECT * FROM Products;
SELECT * FROM Customers;
```

Un programa cliente MySQL como Phpmyadmin o Mysql utiliza el delimitador (;) para separar sentencias y ejecuta cada sentencia por separado. Sin embargo, un procedimiento almacenado consta de varias sentencias separadas por un punto y coma (;).

Si utiliza un programa cliente MySQL para definir un procedimiento almacenado que contiene caracteres de punto y coma, el programa cliente MySQL no tratará todo el procedimiento almacenado como una única sentencia, sino como muchas sentencias independientes.

Por lo tanto, debe redefinir el delimitador temporalmente para poder pasar todo el procedimiento almacenado al servidor como una única sentencia. Para redefinir el delimitador predeterminado, utilice el comando DELIMITER:

```
DELIMITER caracter_delimitador
```

Normalmente, un procedimiento almacenado contiene varias declaraciones separadas por punto y coma (;). Para compilar todo el procedimiento almacenado como una sola declaración compuesta, es necesario cambiar temporalmente el delimitador del punto y coma (;) a otro delimitador como $ $ o //:

```
DELIMITER $$
CREATE PROCEDURE NombreProcedimientoAlmacenado()
BEGIN
-- instrucciones
END $$
DELIMITER ;
```

3.3.- Variables

Una variable es un objeto de datos con nombre cuyo valor puede cambiar durante la ejecución del procedimiento almacenado.

Por lo general, las variables se utilizan en los procedimientos almacenados para almacenar resultados inmediatos. Estas variables son locales para el procedimiento almacenado.

Antes de utilizar una variable, debe declararla.

Declaración de variables

Para declarar una variable dentro de un procedimiento almacenado, utilice la instrucción DECLARE de la siguiente manera:

```
DECLARE nombreVariable tipoDato(tamanio) [DEFAULT valor_por_defecto];
```

En esta sintaxis:

- Primero, especifique el nombre de la variable después de la palabra clave DECLARE. El nombre de la variable debe seguir las reglas de denominación de los nombres de las columnas de la tabla MySQL.
- Segundo, especifique el tipo de datos y la longitud de la variable. Una variable puede tener cualquier tipo de datos MySQL, como INT, VARCHAR y DATETIME.
- Tercero, asigne a una variable un valor predeterminado utilizando la opción DEFAULT. Si declara una variable sin especificar un valor predeterminado, su valor es NULL.

El siguiente ejemplo declara una variable denominada totalVentas con el tipo de datos DEC(10,2) y el valor predeterminado 0.0 de la siguiente manera:

```
DECLARE totalVentas DEC(10,2) DEFAULT 0.0;
```

MySQL le permite declarar dos o más variables que comparten el mismo tipo de datos mediante una única declaración DECLARE. El siguiente ejemplo declara dos variables enteras x e y, y establece sus valores predeterminados en cero.

```
DECLARE x, y INT DEFAULT 0;
```

También es posible declarar una variable sin un valor predeterminado.

```
DECLARE sueldo INT;
```

Asignación de variables

Una vez que se declara una variable, está lista para usarse. Para asignarle un valor a una variable, se utiliza la instrucción SET:

```
SET nombre_variable = valor;
```

Por ejemplo:

```
DECLARE total INT DEFAULT 0;
SET total = 10;
```

El valor de la variable total es 10 después de la asignación. Además de la instrucción SET, puede utilizar la instrucción SELECT INTO para asignar el resultado de una consulta a una variable, como se muestra en el siguiente ejemplo:

```
DECLARE cuentaProducto INT DEFAULT 0;
SELECT COUNT(*) INTO cuentaProducto FROM Productos;
```

En este ejemplo:

1. Primero, se declara una variable denominada cuentaProducto y se inicializa su valor en 0.
2. Luego, utilice la instrucción SELECT INTO para asignar a la variable cuentaProducto la cantidad de productos seleccionados de la tabla de Productos.

El siguiente ejemplo muestra cómo declarar y utilizar una variable en un procedimiento almacenado:

```
DELIMITER $$
CREATE PROCEDURE GetTotalPedidos()
BEGIN
    DECLARE totalPedidos INT DEFAULT 0;
    SELECT COUNT(*) INTO totalPedidos FROM Pedidos;
    SELECT totalPedidos;
END$$
DELIMITER ;
```

3.4.- Parámetros

A menudo, los procedimientos almacenados tienen parámetros. Los parámetros hacen que el procedimiento almacenado sea más útil y reutilizable. Un parámetro en un procedimiento almacenado tiene uno de tres modos: IN, OUT o INOUT. En esta sección, estudiaremos solo los parámetros IN.

IN parámetros

IN es el modo predeterminado. Cuando se define un parámetro IN en un procedimiento almacenado, el programa que lo llama tiene que pasar un argumento al procedimiento almacenado.

Además, el valor de un parámetro IN está protegido. Esto significa que incluso si se cambia el valor del parámetro IN dentro del procedimiento almacenado, su valor original no cambia una vez que el procedimiento almacenado finaliza. En otras palabras, el procedimiento almacenado solo opera en la copia del parámetro IN.

Definición de parámetros

La sintaxis básica para definir un parámetro en procedimientos almacenados es:

```
[IN | OUT | INOUT] nombre_parametro tipoDato[(longitud)]
```

- En primer lugar, especifique el modo del parámetro, que puede ser IN, OUT o INOUT según el propósito del parámetro en el procedimiento almacenado.
- En segundo lugar, especifique el nombre del parámetro. El nombre del parámetro debe seguir las reglas de denominación del nombre de columna en MySQL.
- En tercer lugar, especifique el tipo de datos y la longitud máxima del parámetro.

El siguiente ejemplo crea un procedimiento almacenado que busca todas las oficinas que se encuentran en un país especificado por el parámetro de entrada nombrePais:

```
DELIMITER $$
CREATE PROCEDURE GetOficinasDePais(IN nombrePais VARCHAR(255))
BEGIN
    SELECT * FROM Oficinas WHERE pais = nombrePais
END $$
DELIMITER ;
```

En este ejemplo, nombrePais es el parámetro IN del procedimiento almacenado. Supongamos que quiere encontrar oficinas ubicadas en USA., debe pasar un argumento (USA) al procedimiento almacenado, como se muestra en la siguiente consulta:

```
CALL GetOficinasDePais('USA');
```

3.5.- Funciones almacenadas

Una función almacenada es un tipo especial de programa almacenado que devuelve un único valor.

A continuación se muestra la sintaxis básica para crear una nueva función almacenada:

```
DELIMITER $$
CREATE FUNCTION functionName(param1, param2,…)
RETURNS dataType
BEGIN
    -- instrucciones
    RETURN
END$$
DELIMITER ;
```

En esta sintaxis:

- Primero, especifique el nombre de la función almacenada que desea crear después de las palabras clave CREATE FUNCTION.
- Segundo, enumere todos los parámetros de la función almacenada dentro de los corchetes seguidos del nombre de la función. De manera predeterminada, todos los parámetros son los parámetros IN. No puede especificar modificadores IN, OUT o INOUT para los parámetros.
- Tercero, especifique el tipo de datos del valor de retorno en la declaración RETURNS, que puede ser cualquier tipo de datos MySQL válido.
- Cuarto, escriba el código en el cuerpo de la función almacenada en el bloque BEGIN END. Dentro de la sección del cuerpo, debe especificar al menos una declaración RETURN. La declaración RETURN devuelve un valor a los programas que la llaman. Siempre que se llega a la declaración RETURN, la ejecución de la función almacenada finaliza inmediatamente.

Ejemplo:

```
DELIMITER $$
CREATE FUNCTION GetLimiteCredito(pIdCliente INT) RETURNS DECIMAL(10,2)
BEGIN
    DECLARE credito DECIMAL(10,2) DEFAULT 0;
    SELECT limiteCredito INTO credito FROM Clientes
    WHERE idCliente = pIdCliente;
    RETURN(credito);
END$$
DELIMITER ;
```

3.6.- Instrucciones condicionales

Instrucción IF

La instrucción IF-THEN le permite ejecutar un conjunto de instrucciones SQL en función de una condición especificada. A continuación, se muestra la sintaxis de la instrucción IF-THEN:

```
IF condicion THEN
    --instrucciones;
END IF;
```

En esta sintaxis:

- Primero, especifica una condición para ejecutar el código entre IF-THEN y END IF. Si la condición se evalúa como TRUE, se ejecutarán las instrucciones entre IF-THEN y END IF. De lo contrario, el control pasa a la siguiente instrucción después de END IF.
- Segundo, especifica el código que se ejecutará si la condición se evalúa como TRUE.

Instrucción IF-ELSE

En caso de que desee ejecutar otras instrucciones cuando la condición en la rama IF no evalúe como TRUE, puede utilizar la instrucción IF-THEN-ELSE de la siguiente manera:

```
IF condicion THEN
    --instrucciones;
ELSE
    --instrucciones else;
END IF;
```

Ejemplo:

```
DELIMITER $$
CREATE FUNCTION NivelCliente(credito DECIMAL(10,2))
RETURNS VARCHAR(20)
BEGIN
    DECLARE nivel VARCHAR(20);
    IF credito > 50000 THEN
        SET nivel = 'PLATINUM';
    ELSE
        IF (credito >= 10000) THEN
            SET nivel = 'ORO';
        ELSE
            SET nivel = 'PLATA';
        END IF;
    END IF;
    -- devuelve el nivel del cliente
    RETURN (nivel);
END$$
DELIMITER ;
```

Instrucción CASE

La siguiente es la sintaxis básica de la declaración CASE simple:

```
CASE valor_caso
    WHEN valor1 THEN instrucciones
    WHEN valor2 THEN instrucciones
    ...
    [ELSE instrucciones]
END CASE;
```

En esta sintaxis, la instrucción CASE simple compara secuencialmente el valor_caso con el valor1, valor2, … hasta que encuentra uno que sea igual. Cuando CASE encuentra un valor_caso igual a un valor, ejecuta las instrucciones en la cláusula THEN correspondiente.

Si CASE no puede encontrar ningún valor igual al valor_caso, ejecuta las instrucciones else en la cláusula ELSE, si la cláusula ELSE está disponible.

Instrucciones LOOP

La instrucción LOOP permite ejecutar una o más instrucciones de forma repetida. A continuación, se muestra la sintaxis básica de la instrucción LOOP:

```
[etiqueta_inicio:] LOOP
    --instrucciones
END LOOP [etiqueta_fin]
```

LOOP puede tener etiquetas opcionales al principio y al final del bloque, ejecutando las instrucciones repetidamente. Cada instrucción debe terminar con un delimitador de declaración de punto y coma (;).

Normalmente, se termina el bucle cuando se cumple una condición utilizando la declaración LEAVE.

Esta es la sintaxis típica de la declaración LOOP utilizada con la declaración LEAVE:

```
[etiqueta]: LOOP
    ...
    -- finaliza el loop
    IF condition THEN
        LEAVE [etiqueta];
    END IF;
    ...
END LOOP;
```

Instrucción WHILE

El bucle WHILE es una sentencia que ejecuta un bloque de código repetidamente mientras se cumpla una condición.

Esta es la sintaxis básica de la sentencia WHILE:

```
WHILE condicion DO
    --instrucciones
END WHILE
```

En esta sintaxis:

- Primero, especifica una condición de búsqueda después de la palabra clave WHILE.
- El WHILE comprueba la condición de búsqueda al comienzo de cada repetición.
- Si la condición de búsqueda se evalúa como TRUE, WHILE ejecuta la lista de instrucciones.

WHILE puede ejecutar de 0 a N veces.

Instrucción REPEAT

La instrucción REPEAT ejecuta una o más instrucciones hasta que se cumpla una condición de búsqueda. A continuación, se muestra la sintaxis básica de la instrucción REPEAT:

```
REPEAT
    --instrucciones
UNTIL condicion
```

REPEAT ejecuta las instrucciones hasta que la condición de búsqueda se evalúa como verdadera.

REPEAT comprueba la condición de búsqueda después de la ejecución de la instrucción. Por lo tanto, la instrucción siempre se ejecuta al menos una vez. Por esta razón, REPEAT también se conoce como bucle de post-prueba.

3.8.- Cursores

Para gestionar un conjunto de resultados dentro de un procedimiento almacenado, se utiliza un cursor. Un cursor permite repetir un conjunto de filas devueltas por una consulta y procesar cada fila individualmente.

Primero, declare un cursor mediante la instrucción DECLARE:

```
DECLARE nombre_cursor CURSOR FOR SELECT ...;
```

La declaración del cursor debe ir después de cualquier declaración de variable. Si declara un cursor antes de las declaraciones de variable, MySQL generará un error. Un cursor siempre debe estar asociado a una sentencia SELECT.

A continuación, para usar un cursor utiliza la sentencia OPEN. Esta sentencia inicializa el conjunto de resultados del cursor, por lo que debe ser llamada antes de obtener las filas del conjunto de resultados.

```
OPEN nombre_cursor;
```

Luego, utilice la instrucción FETCH para recuperar la siguiente fila apuntada por el cursor y mover el cursor a la siguiente fila en el conjunto de resultados.

```
FETCH nombre_cursor INTO lista variables;
```

Después, compruebe si hay alguna fila disponible antes de recuperarla. Finalmente, desactive el cursor y libere la memoria asociada mediante la instrucción CLOSE:

```
CLOSE nombre_cursor;
```

Siempre es una buena práctica cerrar un cursor cuando ya no se utiliza.

Al trabajar con el cursor MySQL, también debe declarar un controlador NOT FOUND para manejar la situación en la que el cursor no pudo encontrar ninguna fila.

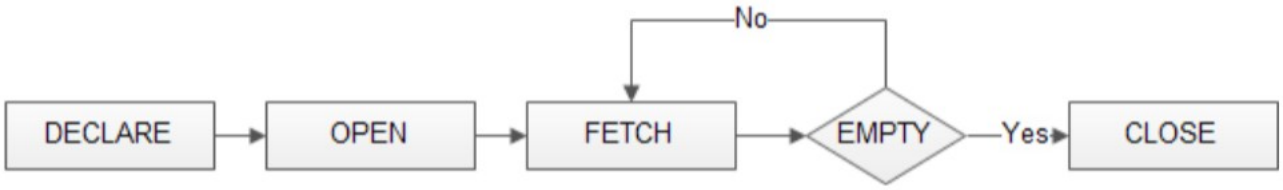
Cada vez que llama a la declaración FETCH, el cursor intenta leer la siguiente fila en el conjunto de resultados. Cuando el cursor llega al final del conjunto de resultados, no podrá obtener los datos y se genera una condición. El controlador se utiliza para manejar esta condición.

Para declarar un controlador NOT FOUND, utilice la siguiente sintaxis:

```
DECLARE CONTINUE HANDLER FOR NOT FOUND SET finished = 1;
```

La variable "finished" indica que el cursor ha llegado al final del conjunto de resultados. Tenga en cuenta que la declaración del controlador debe aparecer después de la declaración de la variable y del cursor dentro de los procedimientos almacenados.

El siguiente diagrama ilustra cómo funciona el cursor MySQL.



Ejemplo:

```
DELIMITER $$
CREATE FUNCTION crearListaEmails( ) RETURNS VARCHAR(4000)
BEGIN
    DECLARE terminado INTEGER DEFAULT 0;
    DECLARE direccionEmail varchar(100) DEFAULT "";
    DECLARE listaEmail varchar(4000) DEFAULT "";
    -- declare cursor for employee email
    DECLARE empleadoEmail CURSOR FOR
        SELECT email FROM Empleado;
    -- declare NOT FOUND handler
    DECLARE CONTINUE HANDLER
        FOR NOT FOUND SET terminado = 1;
    OPEN empleadoEmail;
    getEmail: LOOP --LOOP con etiqueta getEmail
        FETCH empleadoEmail INTO direccionEmail;
        IF terminado = 1 THEN
            LEAVE getEmail;
        END IF;
        -- build email list
        SET listaEmail = CONCAT(direccionEmail,";",listaEmail);
    END LOOP getEmail;
    CLOSE empleadoEmail;
    RETURN(listaEmail);
END$$
DELIMITER ;
```

4.-Triggers

En MySQL, un disparador es un programa almacenado que se invoca automáticamente en respuesta a un evento como insertar, actualizar o eliminar que ocurre en la tabla asociada. Por ejemplo, puede definir un disparador que se invoque automáticamente antes de que se inserte una nueva fila en una tabla.

MySQL admite disparadores que se invocan en respuesta al evento INSERT, UPDATE o DELETE. Un disparador se activa para cada fila que se inserta, actualiza o elimina.

La instrucción CREATE TRIGGER crea un nuevo disparador. Su sintaxis básica es:

```
CREATE TRIGGER nombreTrigger {BEFORE | AFTER} {INSERT | UPDATE| DELETE }
ON nombreTabla FOR EACH ROW
instruccionesTrigger;
```

- Primero, especifica el nombre del disparador que desea crear después de la palabra clave CREATE TRIGGER. Tenga en cuenta que el nombre del disparador debe ser único en la base de datos.
- A continuación, especifica el tiempo de acción del disparador, que puede ser BEFORE o AFTER, lo que indica que el disparador se invoca antes o después de modificar cada fila.
- A continuación, especifica la operación que activa el disparador, que puede ser INSERT, UPDATE o DELETE.
- Después, especifica el nombre de la tabla a la que pertenece el disparador después de la palabra clave ON.
- Finalmente, especifica la sentencia que se ejecutará cuando se active el disparador. Si desea ejecutar varias sentencias, utilice la sentencia compuesta BEGIN END.

El cuerpo del disparador puede acceder a los valores de la columna afectada por la sentencia DML. Para distinguir entre el valor de las columnas BEFORE y ANTES de que se active el DML, utilice los modificadores NEW y OLD.

Por ejemplo, si actualiza la columna "descripcion", en el cuerpo del disparador, puede acceder al valor de la descripción antes de la actualización OLD.descripcion y al nuevo valor NEW.descripcion.

La siguiente tabla ilustra la disponibilidad de los modificadores OLD y NEW:

Evento disparador	OLD	NEW
INSERT	NO	YES
UPDATE	YES	YES
DELETE	YES	NO

Ejemplo:

```
CREATE TRIGGER antes_actualizacion_empleado
BEFORE UPDATE ON Empleados
FOR EACH ROW
    INSERT INTO historial_cambios_empleado
    SET action = 'update',
    idEmpleado = OLD.idEmpleado,
    apellido = OLD.apellido,
    fechaCambio = NOW();
```

4.1.- Consideraciones

Los triggers son una excelente manera de automatizar acciones en la base de datos y asegurar la integridad de los datos. Sin embargo, es crucial que sean utilizados de manera responsable para evitar problemas de rendimiento o complejidad en la gestión de la base de datos.

- **Recursividad**: Si un trigger modifica una tabla que a su vez activa otro trigger, podría entrar en un ciclo infinito.
- **Desempeño**: Los triggers pueden afectar negativamente el rendimiento si no se implementan correctamente, especialmente en bases de datos con un alto volumen de transacciones.
- **Debugging y Mantenimiento**: A diferencia de los procedimientos almacenados estándar o el código en un lenguaje de programación como Python o Java, el código dentro de un trigger puede ser más difícil de depurar y mantener.

4.2.- Uso de triggers

Validaciones complejas

Si bien las restricciones de la base de datos pueden manejar validaciones básicas, como las claves primarias o únicas, los triggers pueden realizar validaciones mucho más complejas. Por ejemplo, puedes usar un trigger para validar que el salario de un empleado no exceda el presupuesto total del departamento al que pertenece.

Manejo de errores

Es fundamental implementar un manejo de errores sólido en tus triggers. Esto ayuda a que el sistema de bases de datos sea más robusto y menos propenso a fallos debidos a datos incorrectos o acciones no deseadas.

Triggers temporales

Algunos DBMS, como SQL Server, ofrecen la posibilidad de crear triggers temporales que se autoeliminan después de un período de tiempo determinado. Esta funcionalidad puede ser útil para situaciones como campañas de marketing o auditorías temporales.

Triggers con múltiples eventos

Es posible definir triggers que se activan con más de un tipo de evento DML (**INSERT**, **UPDATE**, **DELETE**). Esto puede simplificar el código y hacer que la gestión de los triggers sea más eficiente.

Evitar la recursividad

La recursividad en triggers puede causar ciclos infinitos y consumir muchos recursos. Muchos sistemas de gestión de bases de datos permiten desactivar la recursividad en triggers, lo que puede ser una buena práctica para evitar problemas.

Optimización del rendimiento

Debes ser consciente de que los triggers pueden afectar el rendimiento de tu base de datos. Para minimizar este impacto, es recomendable:

- Mantener el código del trigger lo más simple y corto posible.
- Evitar llamar a funciones complejas dentro de triggers.
- Limitar el número de operaciones de lectura y escritura.

Coordinación entre capas

Los triggers realizan operaciones de la capa controlador (código del programa) directamente en el modelo (base de datos). Hay que evitar la redundancia de acciones, definiendo quién y donde realizará cada acción.

Por lo general los triggers realizan funciones transparentes para el programador y se ejecutan directamente en la base de datos. Estas acciones pueden ejecutarse de manera más eficiente directamente sobre la base de datos pero hay que tener en cuenta el tiempo de procesamiento para no saturar el sistema. Si el código del trigger tiene demasiada complejidad algorítmica, seguramente sea porque esa función debe realizarse en la capa controlador.

Una buena documentación nos ayudará a evitar este tipo de fallos.